

Git & GitHub

1) What is Git?

Ans : - Git is a local version control system, a command line tool installed on your computer that tracks changes in your code.

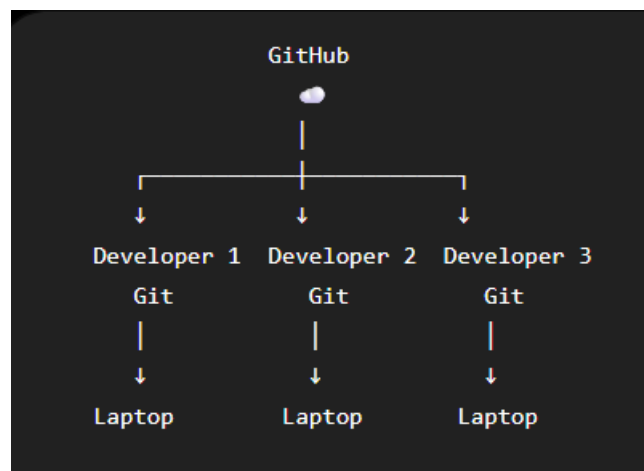
- Example: Say you're working on an app and pushing new features every single day. If the app suddenly breaks, how do you know what went wrong? That's exactly why we use Git. It saves your progress step-by-step. If your latest code introduces an error, you can just jump back to your previous working code without losing anything.
- It works on offline mode and if we have to push on GitHub that time we need to go online.

2) What is GitHub?

Ans: - GitHub is an online platform that hosts Git repositories and helps developers store, share, collaborate on, and manage code.

3) How does git and GitHub work together?

Ans: -



4) How do I view the code changes in a GitHub commit or Pull Request, including which lines were added and deleted?

Ans :- So, GitHub uses:

- **Red** → code that was **deleted/removed** (- old code)
- **Green** → code that was **added** (+ new code)

Some important commands for Git

ls - It lists the files and folders inside your current directory.

cd - Change my current folder/location (change Directory)

cd .. – Go one folder(directory) back

cd ../../.. – Go back 3 folders back.

cd foldername – Going inside foldername.

mkdir - create a new folder/directory.

git init – ‘git init’ turns a normal folder into a Git repository by creating the ‘.git’ directory and preparing Git to track your project, but it does not save your files until you ‘git add’ and ‘git commit’ them.

ls -a - lists all files and folders, including hidden ones like ‘.git’.

touch - create a file.

git status - shows the current state of your Git repository, what files are changed, new, deleted, or staged.

rd – remove/delete directory (Folder)

rm – remove/delete a file

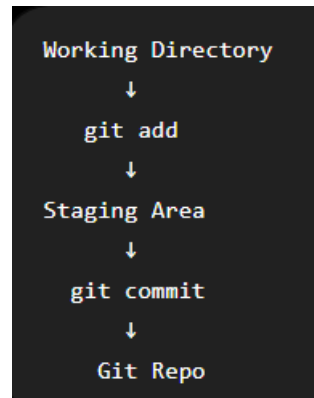
-rf – remove/ delete forcefully

-r – Remove content by going each and every folder

rm -rf - Delete a folder + everything inside forcefully

```
.    = current directory
..   = parent directory
```

git add - puts your changed or new files into Git's staging area, preparing them for the next commit.



5) Git Staging Area :

Ans :- The Staging Area is a place where you tell Git:
These are the exact changes I want in my next commit.

```

You changed 3 files:
Hello.java      modified
Calculator.java modified
notes.txt       new

But you only want Hello.java in your next commit.

</> Bash
git add Hello.java

Now:

Staged:
Hello.java      ✓

Not staged:
Calculator.java ✗
notes.txt       ✗
  
```

-m - Message label (To show message)

For Data insertion in txt file

vi [filename] – To open vim editor and add text / content in that file.

Esc and then **:wq** = save + exit

Esc and then **:q!** = don't save + exit [Also if we are in read-only mode]

Command	What it does
cat file.txt	Show file contents
cat > file.txt	Create/overwrite file
cat >> file.txt	Add content to the end
vi file.txt	Open file in vi editor

6) What is a commit hash?

Ans:-

When you run:

```
</> Bash
```

```
git commit -m "Added munna file"
```

Git creates a **commit** and gives that commit a unique ID called a **commit hash**.

Example:

```
[master (root-commit) 7a3f21c] Added munna file
```

Here:

```
7a3f21c
```

git log – shows the commit full hash code and the details of commit like who did (author), and when with time and message what committed.

git log –oneline – It shows short hash code (one line short commit hash code).

git reset <hash> - It will remove the

7) What is branch and head?

Ans:-

Branch: The line of development where I make and commit my changes.

HEAD: A pointer that indicates where I am currently in the Git history, usually the current branch.